

GOOGLE CAREER PREPARATION

DAY 1 — DETAILED ACTION PLAN

Thursday, 10 September 2026

The purpose of Day 1 is to establish the engineering environment, start your DSA discipline, and begin the first real project. The Python work is deliberately concrete: you will build a small Document Management Service that prepares the business logic used later by Project 1.

Three tracks for Day 1

Track	Tomorrow's deliverable
DSA	Basics + 3 problems
Python	Build a small Document Management Service in pure Python
Project 1	Base skeleton + FastAPI API

1. DSA — Basics + 3 Problems

Time: 45 minutes

Your first objective is **not problem count**. It's learning how to recognize the pattern.

Learn / revise

- Arrays
- Hash maps
- Hash sets
- Time complexity
- Space complexity
- Two-pointer concept
- Frequency counting

You should understand:

Array lookup	→ $O(1)$ average for index
HashMap lookup	→ $O(1)$ average
HashMap insertion	→ $O(1)$ average
Sorting	→ $O(n \log n)$
Linear scan	→ $O(n)$
Nested scan	→ $O(n^2)$

Solve exactly these 3 problems

• Problem 1 — Two Sum

Focus on: Brute force → $O(n^2)$; HashMap → $O(n)$

• Problem 2 — Contains Duplicate

Focus on: Array → Set

• Problem 3 — Valid Anagram

Focus on: frequency counting; HashMap

For EVERY problem record:

Problem:
Pattern:
Brute-force approach:
Optimized approach:
Time complexity:
Space complexity:
Why does the optimized approach work?
What mistake did I make?

DSA deliverable

Create:

```
google-prep/  
  dsa/  
    arrays-hashmaps/  
      two_sum.py  
      contains_duplicate.py  
      valid_anagram.py
```

And:

```
dsa-notes.md
```

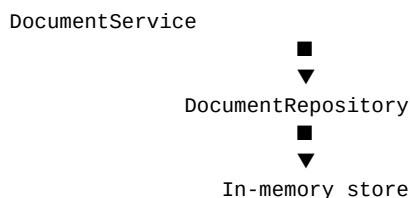
Target: 3 problems completed properly. Don't do 10 problems just to increase the number.

2. PYTHON — Build a Document Management Service

Time: 40 minutes

The Python portion is not just “learn Python.” You are going to build a **mini Document Management Service**, but **without FastAPI**. Think of it as building the business logic first.

The architecture will be:



Create this structure:

```
python/  
  document_service/  
    document.py  
    repository.py  
    service.py  
    exceptions.py  
    test_document_service.py
```

The point is to practice **Python software engineering**, not build a useful application yet.

2.1 Document

Create a Python class representing a document. It should contain:

- id
- name
- document_type
- version
- status

For example:

```
Document(  
    id="doc-001",  
    name="engine-requirement",  
    document_type="requirement",  
    version="1.0",  
    status="created"  
)
```

What you're practicing:

- Classes
- Constructors
- Type hints
- Data modelling
- `__repr__`
- validation

2.2 DocumentRepository

Create a repository that stores documents **in memory**. Conceptually:

```
DocumentRepository  
■  
■■■ create()  
■■■ get()  
■■■ list()  
■■■ delete()
```

Internally:

```
documents = {}
```

For example:

```
documents["doc-001"] = Document(...)
```

Implement:

```
create(document)  
get(document_id)  
list()  
delete(document_id)
```

What you're practicing:

- Separating business logic from data storage
- Repository pattern
- Designing code so the storage implementation can later change from an in-memory dictionary to PostgreSQL without rewriting the entire application.

2.3 DocumentService

Create the business layer. Architecture:

```
DocumentService  
■  
▼  
DocumentRepository
```

Implement:

```
create_document()  
get_document()  
list_documents()  
delete_document()
```

Add basic business rules:

- name must not be empty
- version must exist
- document_type must be valid

If the document doesn't exist:

`DocumentNotFoundError`

2.4 Custom exceptions

Create:

`exceptions.py`

with:

- `DocumentNotFoundError`
- `InvalidDocumentError`

You're learning that production applications shouldn't simply do a generic `raise Exception(...)`; use meaningful, specific exceptions.

2.5 Unit tests

This is the most important part of the Python exercise. Create:

`test_document_service.py`

Test at least:

- **Test 1 — Create document:** create → document returned
- **Test 2 — Retrieve document:** create → get → same document
- **Test 3 — Retrieve nonexistent document:** get unknown ID → `DocumentNotFoundError`
- **Test 4 — Delete document:** create → delete → get fails
- **Test 5 — Invalid document:** empty name → `InvalidDocumentError`

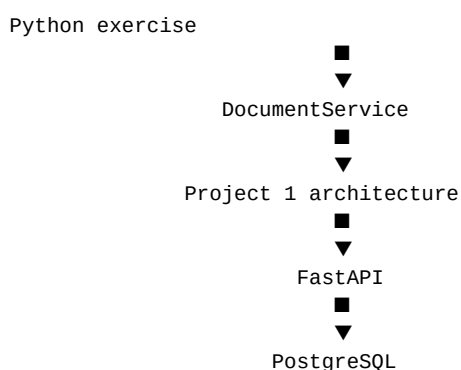
So tomorrow you'll have **at least 5 tests**.

2.6 What this teaches you

Python

-
- Classes
- Type hints
- Data structures
- Dictionaries
- Exceptions
- Separation of concerns
- Repository pattern
- Service layer
- Unit testing
- Clean code

And most importantly:



So the Python exercise **isn't disconnected homework**. It's the first version of the core component you'll put behind the API in Project 1.

3. PROJECT 1 — Base Skeleton + FastAPI

Time: 45 minutes

Today you are **not building RAG**. You are building the foundation.

Create the main GitHub repository:

```
ai-engineering-intelligence-platform
```

Create:

```
ai-engineering-intelligence-platform/
■
■■■ README.md
■■■ docs/
■■■ src/
■■■ tests/
■■■ scripts/
■■■ configs/
■■■ .gitignore
■■■ requirements.txt
■■■ LICENSE
```

Create a FastAPI application. Target structure:

```
ai-engineering-intelligence-platform/
■
■■■ src/
■   ■■■ main.py
■   ■■■ api/
■   ■■■ services/
■   ■■■ models/
■   ■■■ repositories/
■   ■■■ core/
■
■■■ tests/
■■■ docs/
■■■ requirements.txt
■■■ README.md
```

Create:

```
GET /
```

Response:

```
{
  "application": "AI Engineering Intelligence Platform",
  "status": "running",
  "version": "0.1.0"
}
```

Then create:

```
GET /health
```

Response:

```
{
  "status": "healthy"
}
```

3.1 Your First API

Create the beginning of:

```
POST /documents
```

For Day 1, **don't actually process PDFs yet**.

Just accept:

```
{
  "name": "example-requirement",
  "document_type": "requirement",
  "version": "1.0"
}
```

and return something like:

```
{
  "id": "generated-id",
  "name": "example-requirement",
  "status": "created"
}
```

This gives us the first vertical slice:

```
Client
  ↓
FastAPI
  ↓
Document API
  ↓
Response
```

4. Git Discipline

This is important. Don't wait six months and then upload everything to GitHub.

Start behaving like you're working on a production project.

Make your first commit:

```
feat: initialize AI engineering intelligence platform
```

Then:

```
feat: add FastAPI application skeleton
```

Then:

```
feat: add document creation endpoint
```

Your Git history itself should tell the story of the project.

5. Architecture Journal

Time: 20 minutes

Create:

```
docs/architecture-decisions.md
```

Write your first three decisions.

Decision 1 — Why Python?

Example: Python is selected as the primary implementation language because the platform requires extensive ML, NLP, LLM and data-processing capabilities while also supporting backend API development.

Decision 2 — Why FastAPI?

Explain:

- Python ecosystem
- API development
- type validation
- async support

Decision 3 — Why PostgreSQL eventually?

Explain:

- relational metadata
- transactions
- mature ecosystem
- structured engineering information

These documents will become valuable later when we practice **system design interviews**.

6. Day-1 Final Checklist

DSA

- Learned/revised arrays
- Learned/revised hash maps
- Understood Big-O
- Solved Two Sum
- Solved Contains Duplicate
- Solved Valid Anagram
- Documented approaches
- Added solutions to GitHub

Python

- Created Python project structure
- Created Document class
- Added type hints
- Created DocumentRepository
- Implemented create/get/list/delete
- Created DocumentService
- Added validation rules
- Created custom exceptions
- Installed pytest
- Written at least 5 tests

Project 1

- Created GitHub repository
- Created README
- Created project structure
- Created FastAPI application
- ``/`` endpoint works
- ``/health`` endpoint works
- ``POST /documents`` works
- Added Git commits

Engineering

- Created architecture decision document
- Documented why Python
- Documented why FastAPI
- Documented why PostgreSQL

7. What you should NOT do tomorrow

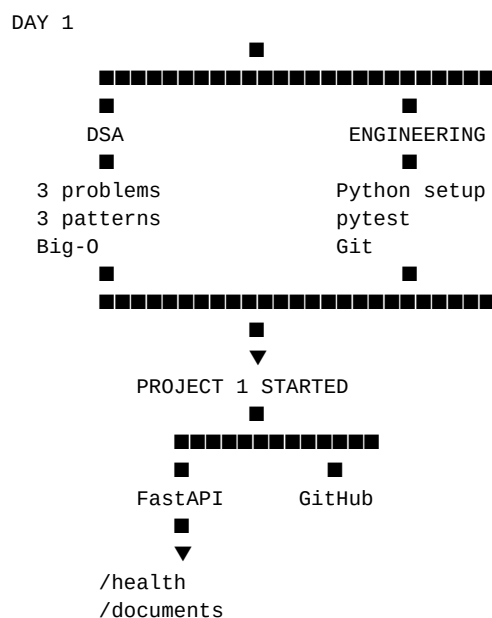
This is equally important.

- Start learning LangChain
- Start building an agent
- Start a chatbot
- Start Kubernetes
- Start GCP
- Watch 5 hours of YouTube tutorials
- Solve 15 random LeetCode problems
- Spend 2 hours designing a fancy UI
- Try to finish Project 1

Tomorrow is about laying the foundation.

8. Your Day-1 Definition of DONE

At approximately **2½ hours**, you should have:



■ Day-1 success metric

Don't measure success by hours. Measure it by:

3 DSA problems + working FastAPI service + GitHub repository + tests + architecture notes.

If you complete those, **Day 1 is a successful start to the 12-month plan.**

9. The Rule for the Entire Year

Whenever you finish a day's work, record four things:

```

Date:
Hours:
what I built:
what I learned:
what I struggled with:
Tomorrow's first task:
  
```

That last line is particularly important.

It means you never start the next day wondering:

"What should I work on today?"

You simply continue.

10 September 2026 is Day 1. The objective isn't to become Google-ready tomorrow. The objective is to make sure Day 365 looks radically different from Day 1.